Lecture Notes for

Neural Networks I

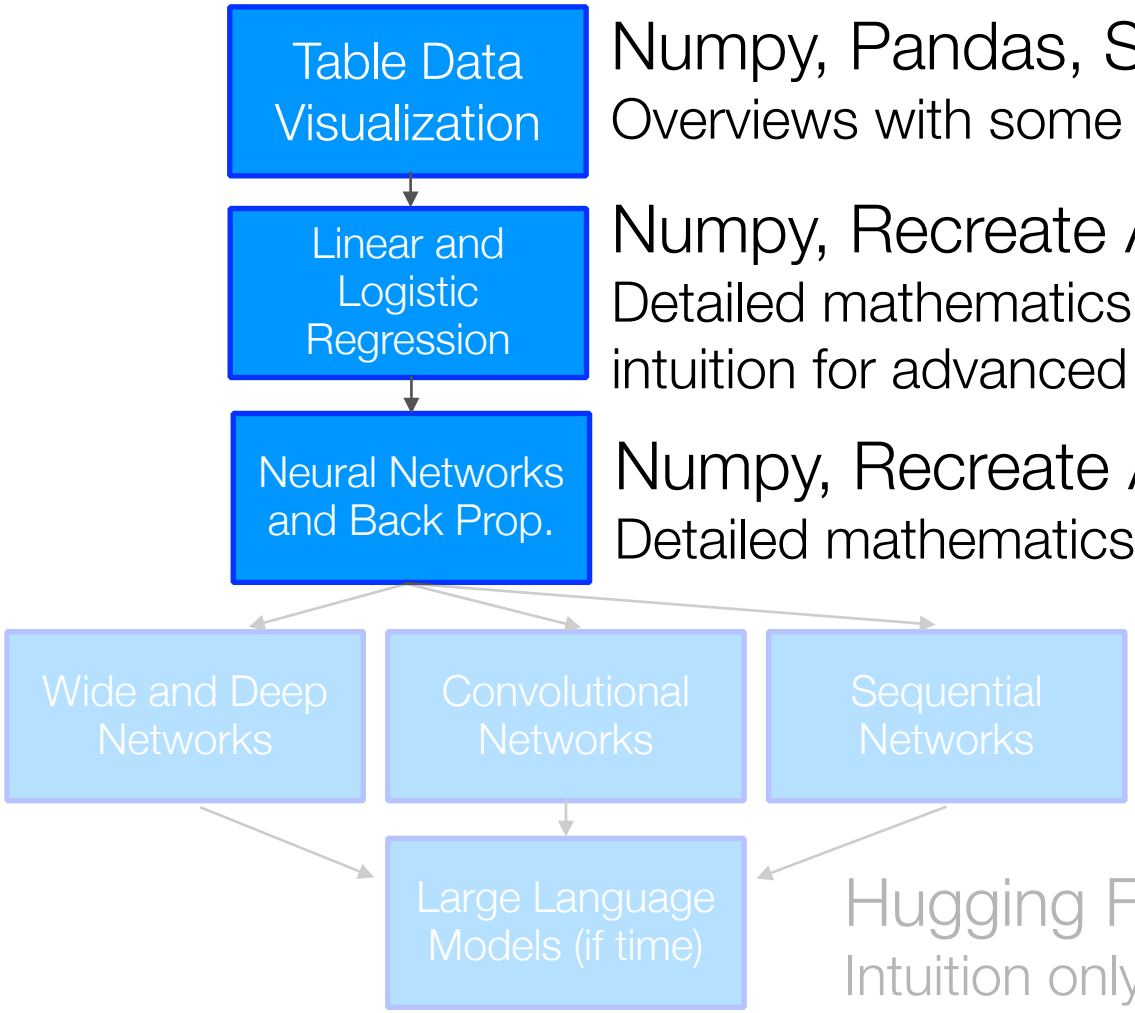


Neural Network Optimization and Activation

Class Logistics and Agenda

- Logistics
  - grading update
- Agenda:
  - Optimization techniques and programming:
    - ~~API Basics, Auto-differentiation~~
    - ~~Momentum and Learning rates~~
    - Objective Functions (started last time)
    - Weight Initialization
    - More activations: Tanh, ReLU, SiLU
    - Adaptive learning: AdaGrad, AdaM, etc.

Class Overview, by topic



Numpy, Pandas, Seaborn
Overviews with some in-depth discussion

Numpy, Recreate API for Scikit-learn
Detailed mathematics for simple optimization
intuition for advanced optimization

Numpy, Recreate API for DL Frameworks
Detailed mathematics for NN operations

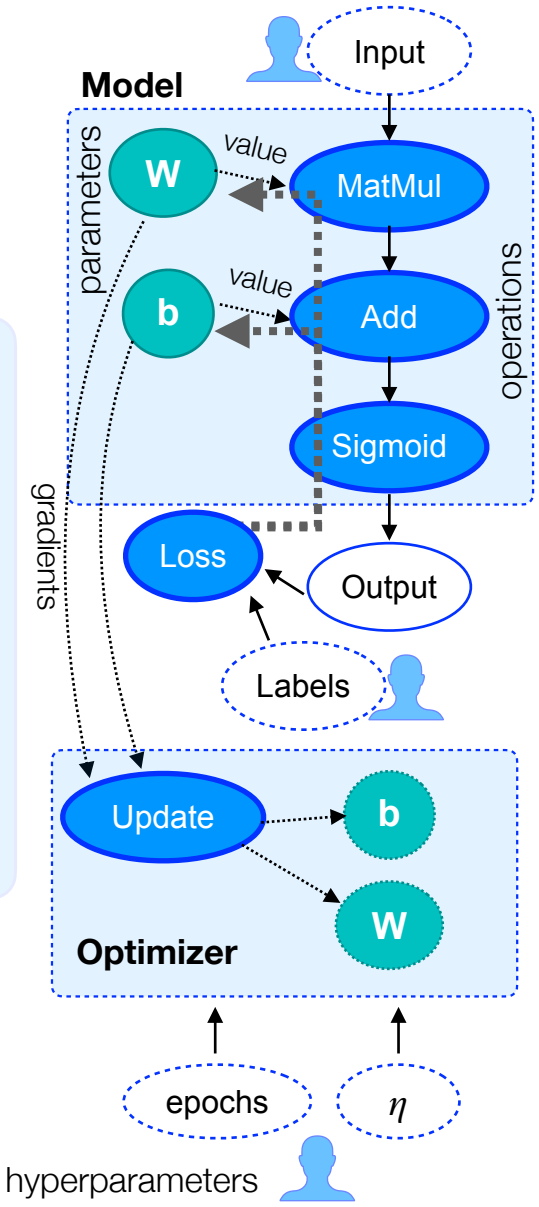
Hugging Face
Intuition only

API Building, Thus Far:

```
# create model
model = Model([Dense(64,32), Sigmoid(),
               Dense(32,10), Sigmoid()])

# set loss function
mse_loss = MSELoss()
opt = SGD(model.parameters,
          learning_rate=1.0, momentum=0.1,
          decay_rate=0.9, step_size=100)

for epoch in range(1000):
    train_step(model, opt, mse_loss,
      X_train, y_train_ohe, batch_size=128)
```

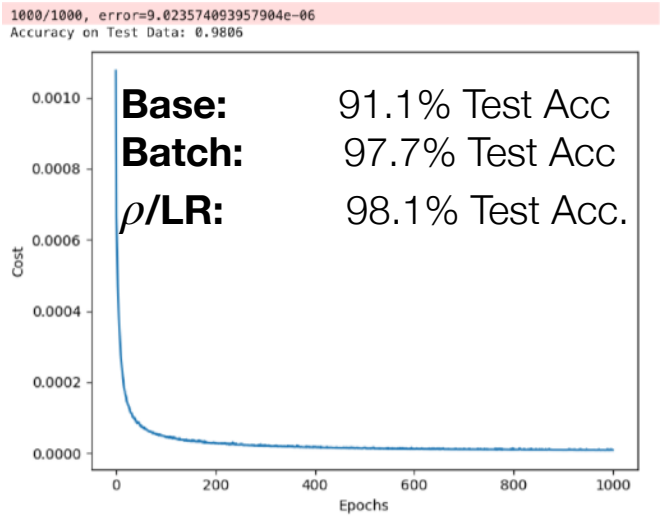


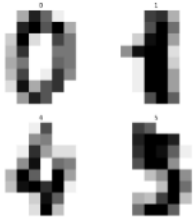
Our API allows us to add many operations that can track their gradients

We can now have full control over loss functions, optimizers, learning rate schedules, etc. via simple sub-classing!

Demo

LarsonUtils.py
07. Practical Neural Networks API.ipynb





Digits: Saturated performance, 98%

```
# create model
model = Model([Dense(64,32), Sigmoid(),
               Dense(32,10), Sigmoid()])

# set loss function
mse_loss = MSELoss()
opt = SGD(model.parameters, learning_rate=1.0, momentum=0.1,
                decay_rate=0.9, step_size=100)

for epoch in range(1000):
    train_step(model, opt, mse_loss, X_train, y_train_ohe,
                batch_size=128)
```

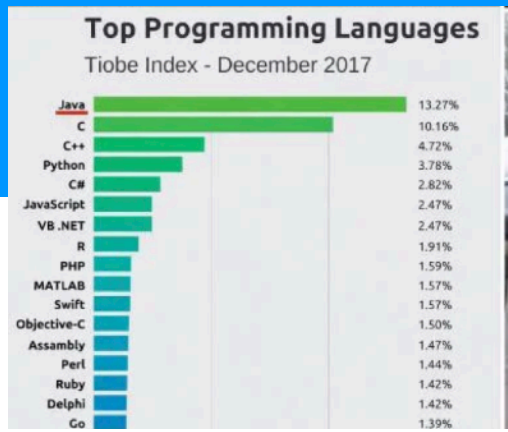
New Dataset: **Fashion MNIST**



- larger images
- still 10 classes
- still flatten pixels for table data
- hard to get more than 85%

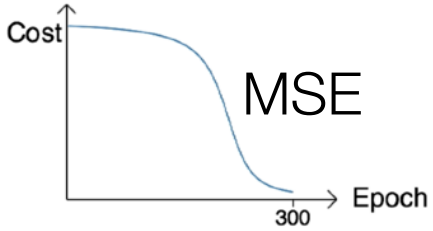
Objective Function



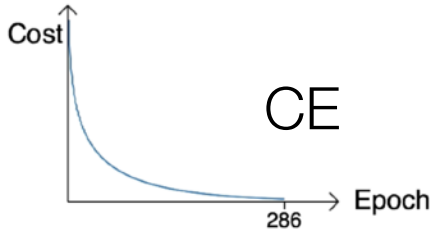




$$J(\mathbf{W}) = \sum_i \left(\mathbf{y}^{(i)} - [\mathbf{a}^{(L+1)}]^{(i)}\right)^2$$



$$J(\mathbf{W}) = -\sum_i \left[\mathbf{y}^{(i)} \ln([\mathbf{a}^{(L+1)}]^{(i)}) + (1 - \mathbf{y}^{(i)}) \ln(1 - [\mathbf{a}^{(L+1)}]^{(i)})\right]$$



Practical Implementation of Architectures

$$J(\mathbf{W}) = -\left[\mathbf{y}^{(i)} \ln([\mathbf{a}^{(L+1)}]^{(i)}) + (1-\mathbf{y}^{(i)})\ln(1-[\mathbf{a}^{(L+1)}]^{(i)})\right]$$

likely to speed up initial training

$$\left[\frac{\partial J(\mathbf{W})}{\partial \mathbf{z}^{(L)}}\right]^{(i)} = -\frac{\partial}{\partial \mathbf{z}^{(L)}}\left[\mathbf{y}^{(i)} \ln([\mathbf{a}^{(L+1)}]^{(i)}) + (1-\mathbf{y}^{(i)})\ln(1-[\mathbf{a}^{(L+1)}]^{(i)})\right]$$

only $\mathbf{a}$ has dependence on $\mathbf{z}$

gradient per instance, i

$$= -\left[\mathbf{y}^{(i)}\frac{\partial}{\partial \mathbf{z}^{(L)}}\left(\ln([\mathbf{a}^{(L+1)}]^{(i)})\right) + (1-\mathbf{y}^{(i)})\frac{\partial}{\partial \mathbf{z}^{(L)}}\left(\ln(1-[\mathbf{a}^{(L+1)}]^{(i)})\right)\right]$$

$$= -\left[\mathbf{y}^{(i)}\frac{1}{[\mathbf{a}^{(L+1)}]^{(i)}}\left(\frac{\partial}{\partial \mathbf{z}^{(L)}}[\mathbf{a}^{(L+1)}]^{(i)}\right) + \frac{(1-\mathbf{y}^{(i)})}{1-[\mathbf{a}^{(L+1)}]^{(i)}}\left(-\frac{\partial}{\partial \mathbf{z}^{(L)}}[\mathbf{a}^{(L+1)}]^{(i)}\right)\right]$$

$$= -\left[\mathbf{y}^{(i)}\frac{1}{[\mathbf{a}^{(L+1)}]^{(i)}}\left([\mathbf{a}^{(L+1)}]^{(i)}(1-[\mathbf{a}^{(L+1)}]^{(i)})\right) - \frac{(1-\mathbf{y}^{(i)})}{1-[\mathbf{a}^{(L+1)}]^{(i)}}\left([\mathbf{a}^{(L+1)}]^{(i)}(1-[\mathbf{a}^{(L+1)}]^{(i)})\right)\right]$$

$$= -\left[\mathbf{y}^{(i)}\left(1-[\mathbf{a}^{(L+1)}]^{(i)}\right) - (1-\mathbf{y}^{(i)})\left([\mathbf{a}^{(L+1)}]^{(i)}\right)\right]$$

$$= -\left[\mathbf{y}^{(i)} - \mathbf{y}^{(i)}[\mathbf{a}^{(L+1)}]^{(i)} - [\mathbf{a}^{(L+1)}]^{(i)} + [\mathbf{a}^{(L+1)}]^{(i)}\mathbf{y}^{(i)}\right] \quad = [\mathbf{a}^{(L+1)}]^{(i)} - \mathbf{y}^{(i)}$$

$$\mathbf{V}^{(2)} = -2(\mathbf{Y}-\mathbf{A}^{(3)}) \odot \mathbf{A}^{(3)} \odot (1-\mathbf{A}^{(3)})$$ old update

Cross Entropy

$$J(\mathbf{W}) = -\left[\mathbf{y}^{(i)} \ln([\mathbf{a}^{(L+1)}]^{(i)}) + (1-\mathbf{y}^{(i)})\ln(1-[\mathbf{a}^{(L+1)}]^{(i)})\right]$$

likely to speed up initial training

$$\left[\frac{\partial J(\mathbf{W})}{\partial \mathbf{z}^{(L)}}\right]^{(i)} = [\mathbf{a}^{(L+1)}]^{(i)} - \mathbf{y}^{(i)} \quad \Rightarrow_{\forall i} \frac{1}{M}\sum_{i}\left([\mathbf{a}^{(L+1)}]^{(i)} - \mathbf{y}^{(i)}\right)$$

across all instances, i

$$= \underbrace{\left[\frac{\partial \mathbf{a}^{(L+1)}}{\partial \mathbf{z}^{(L)}}\right]^{(i)}}_{\texttt{sigmoid.backward(…)}} \cdot \underbrace{\left[\frac{\partial J(\mathbf{W})}{\partial \mathbf{a}^{(L+1)}}\right]^{(i)}}_{\texttt{loss.backward(…)}}$$

we only get this simplification by **combining** gradient calculation of cross entropy and final sigmoid… but our API keeps them separate…

```
class CategoricalCrossEntropy(Loss):
    def __call__(self, y_true, y_pred):
        self.y_true = y_true
        self.y_pred = y_pred
        self.output = -np.mean(y_true * np.log(self.y_pred))
        return self.output

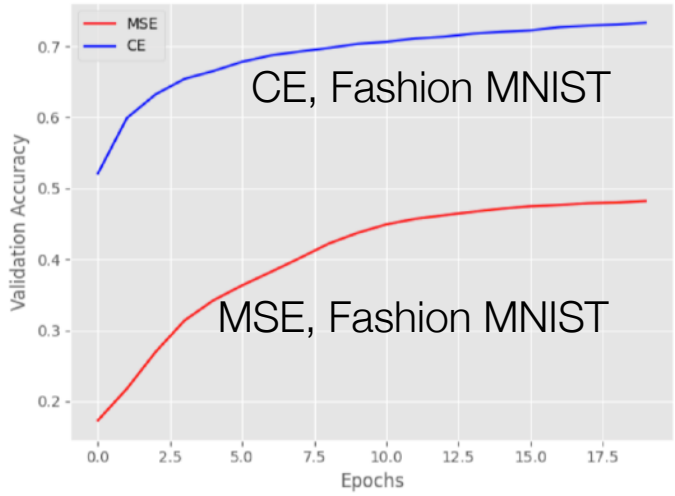
    def backward(self):
        return (self.y_pred - self.y_true) / batch_size

model_ce = Model([Dense(784,256), Sigmoid(),
                  Dense(256,128), Sigmoid(),
                  Dense(128,10)])
# set loss function
ce_loss = CategoricalCrossEntropy()
opt = SGD(model_ce.parameters, learning_rate=0.01, momentum=0.1, decay_rate=0.9, step_size=10)
train_model(model_ce, ce_loss, opt, epochs=20, print_every=5)
```

assume logits as input (before sigmoid)

use simplified **A-Y**

no explicit sigmoid on final layer



Practical Initialization of Architectures

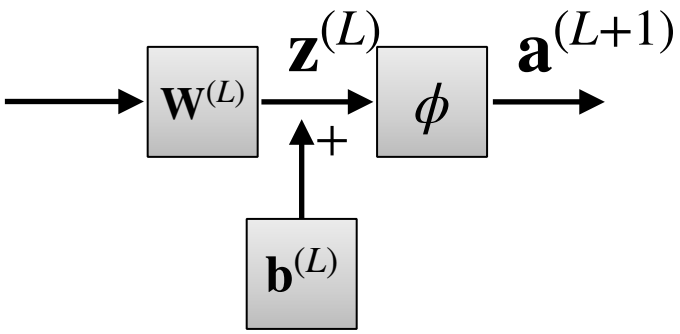


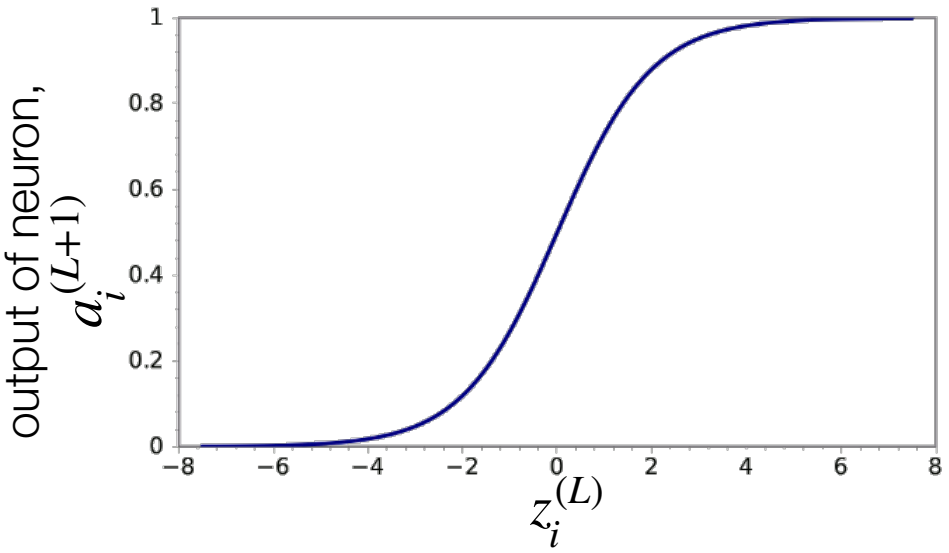
Formative Self Test

- for adding Gaussian random variables, variances add together

$\mathbf{a}^{(L+1)} = \phi\left(\mathbf{W}^{(L)} \cdot \mathbf{a}^{(L)} + \mathbf{b}^{(L)}\right)$ assume each element of $\mathbf{a}^{(L)}$ is Gaussian

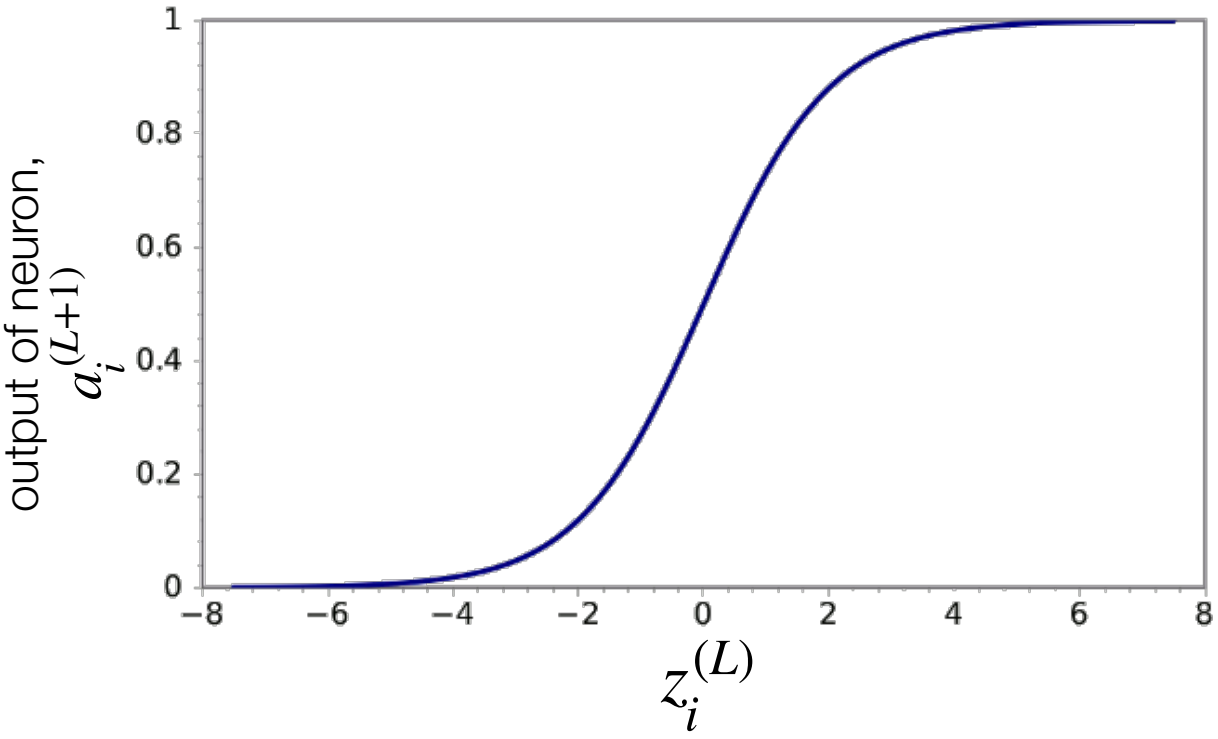
- If you initialized the weights, $\mathbf{W}$, with too large variance, you would expect the output of the neuron, $\mathbf{a}^{(L+1)}$, to be:
  - A. saturated to "1"
  - B. saturated to "0"
  - C. could either be saturated to "0" or "1"
  - D. would not be saturated





Formative Self Test

- What is the derivative of a saturated sigmoid neuron?
  - A. zero
  - B. one
  - C. $a \times (1 - a)$
  - D. it depends

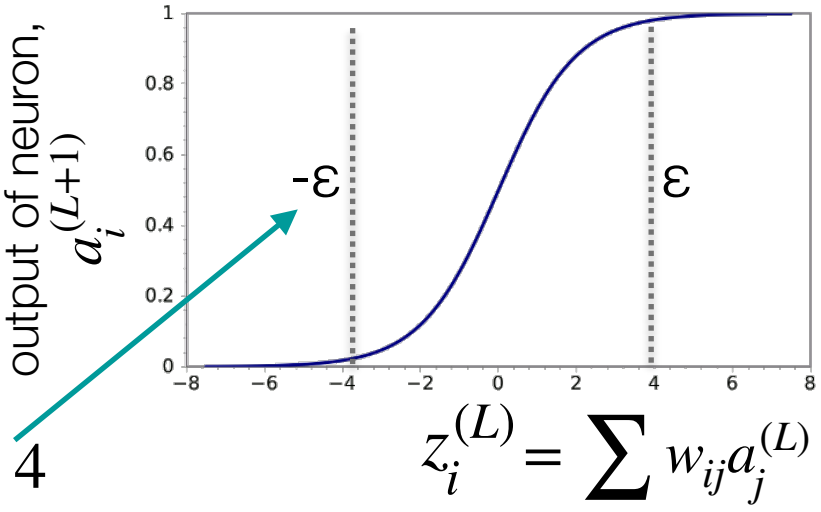


Practical Implementation of Architectures

Weight initialization: try not to saturate!

$$\mathbf{z}^{(L)} = \mathbf{W}^{(L)}\mathbf{a}^{(L)} \qquad z_i^{(L)} = \sum_j^{n^{(L)}} w_{ij} a_j^{(L)}$$

z is a weighted sum of ith row



- want $-\epsilon < z_i^{(L)} < \epsilon$ (no saturation) $\epsilon \approx 4$
- $\therefore \text{Std}[z_i^{(L)}] \approx 4 \implies \text{Std}[a_i^{(L+1)}] \approx 1$
  - $a_i^{(L+1)}$ will be well distributed on interval [0,1]
- since $z_i^{(L)} = \sum w_{ij} a_j^{(L)}$, we want to "squash" starting weights in row i
- how do we squash w_{ij}, such that $\text{Std}[z_i^{(L)}] \approx 4$?
- **One solution**: draw w_{ij} from a Normal Distribution, $w_{ij} \sim \mathcal{N}(0,\alpha)$
  - solve for a good α such that **Goal**: $\text{Std}[z_i^{(L)}] \approx 4$

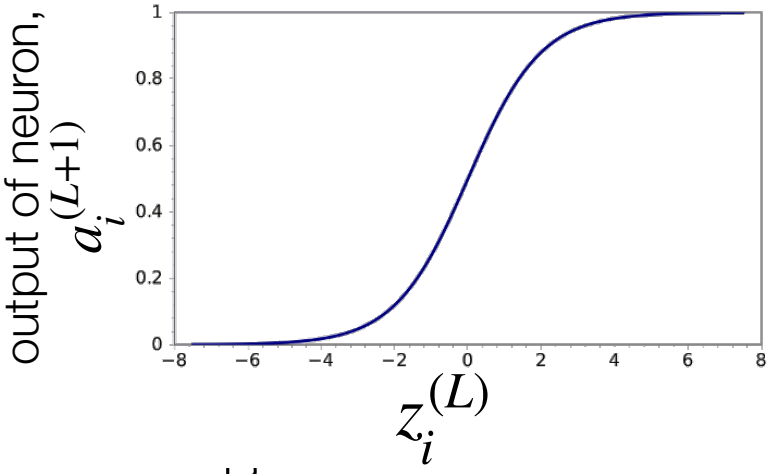
Initialization, check for understanding

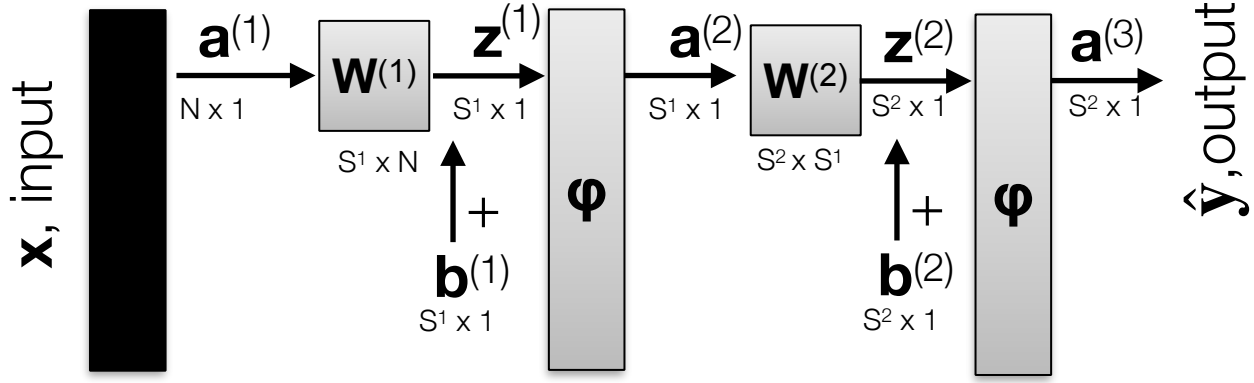
Goal: $\text{Std}[z_i^{(L)}] \approx 4$

$\text{Std}[a_i^{(L+1)}] \approx 1$

Assume:

$\text{Std}[a_i^{(L)}] \approx 1$ $\quad w_{ij} \sim \mathcal{N}(0,\alpha)$





$z_i^{(L)}$

$a_i^{(L)}$

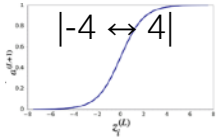
w_{ij}

} want all zero mean to start with…

$\mathbf{b}^{(L)}$

help make zero mean

Glorot Weight Initialization



try not to saturate

$$z_i^{(L)} = \sum_j^{n^{(L)}} w_{ij} a_j^{(L)} \qquad w_{ij} \sim \mathcal{N}(0,\alpha) \qquad \textbf{Goal: } \text{Std}[z_i^{(L)}] \approx 4$$

Random variables add variances: $\text{Var}[z_i^{(L)}] = \sum_j^{n^{(L)}} \text{Var}[w_{ij} \cdot a_j^{(L)}]$

$$\text{Var}[z_i^{(L)}] = \sum_j^{n^{(L)}} E[w_{ij}]^2 \text{Var}[a_j^{(L)}] + \text{Var}[w_{ij}] E[a_j^{(L)}]^2 + \text{Var}[w_{ij}]\text{Var}[a_j^{(L)}] = \sum_j^{n^{(L)}} \text{Var}[w_{ij}]\text{Var}[a_j^{(L)}]$$

…both are zero mean…

same var, $\forall j$

$$\text{Var}[z_i^{(L)}] = n^{(L)}\text{Var}[w_{ij}]\text{Var}[a^{(L)}] = n^{(L)}\text{Var}[w_{ij}]$$

≈ 1 , from previous layer

$$\text{Std}[z_i^{(L)}] = \sqrt{n^{(L)}} \cdot \text{Std}[w_{ij}]$$

$$\text{Std}[z_i^{(L)}] = 4 = \sqrt{n^{(L)}} \cdot \text{Std}[w_{ij}]$$

$$\text{Std}[w_{ij}] = 4 \cdot \sqrt{\frac{1}{n^{(L)}}}$$

$$w_{ij}^{(L)} \sim \mathcal{N}\left(0,\ 4 \cdot \sqrt{\frac{1}{n^{(L)}}}\right)$$ forward from sigmoid

Understanding the difficulty of training deep feedforward neural networks

Xavier Glorot and Yoshua Bengio, JMLR 2010

Glorot Weight Initialization

$$\text{Std}[z_i^{(L)}] = 4 = \sqrt{n^{(L)}} \cdot \text{Std}[w_{ij}]$$

$$w_{ij}^{(L)} \sim \mathcal{N}\left(0, 4 \cdot \sqrt{\frac{1}{n^{(L)}}}\right)$$ forward from sigmoid

$$\mathbf{v}^{(L)} = \mathbf{a}^{(L)}(1 - \mathbf{a}^{(L)})\mathbf{W}^{(L)} \cdot \mathbf{v}^{(L+1)}$$

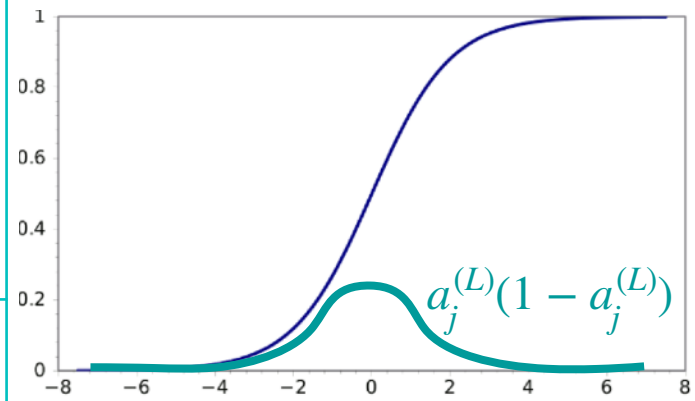
want to keep variance of $\mathbf{v}$ stable magnitude → stable gradient

Similar calculation for back prop.

$$\text{Var}[v_i^{(L)}] = n^{(L+1)}\text{Var}[w_{ij}]\text{Var}[v_j^{(L+1)} \cdot a_j^{(L)}(1 - a_j^{(L)})]$$

$$\text{Std}[v_i^{(L)}] = \sqrt{n^{(L+1)}} \cdot \text{Std}[w_{ij}] \cdot 0.25$$ want = 1

$$w_{ij}^{(L)} \sim \mathcal{N}\left(0, 4 \cdot \sqrt{\frac{1}{n^{(L+1)}}}\right)$$ backward from sensitivity



$$w_{ij}^{(L)} \sim \mathcal{N}\left(0, 4 \cdot \sqrt{\frac{2}{n^{(L)} + n^{(L+1)}}}\right)$$

compromise

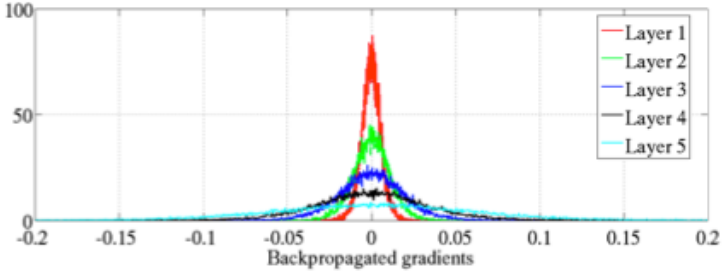
$$w_{ij}^{(L)} \sim U\left[\pm 4\sqrt{\frac{6}{n^{(L)} + n^{(L+1)}}}\right]$$

if drawn from uniform dist.

Glorot Weight Initialization

Understanding the difficulty of training deep feedforward neural networks

Xavier Glorot **Yoshua Bengio**
DIRO, Université de Montréal, Montréal, Québec, Canada



Starting gradient histograms per layer

standard initialization

Figure 7: *Back-propagated gradients normalized histograms with hyperbolic tangent activation, with standard (top) vs normalized (bottom) initialization. Top: 0-peak decreases for higher layers.*

Beyond Sigmoid: Other Activations

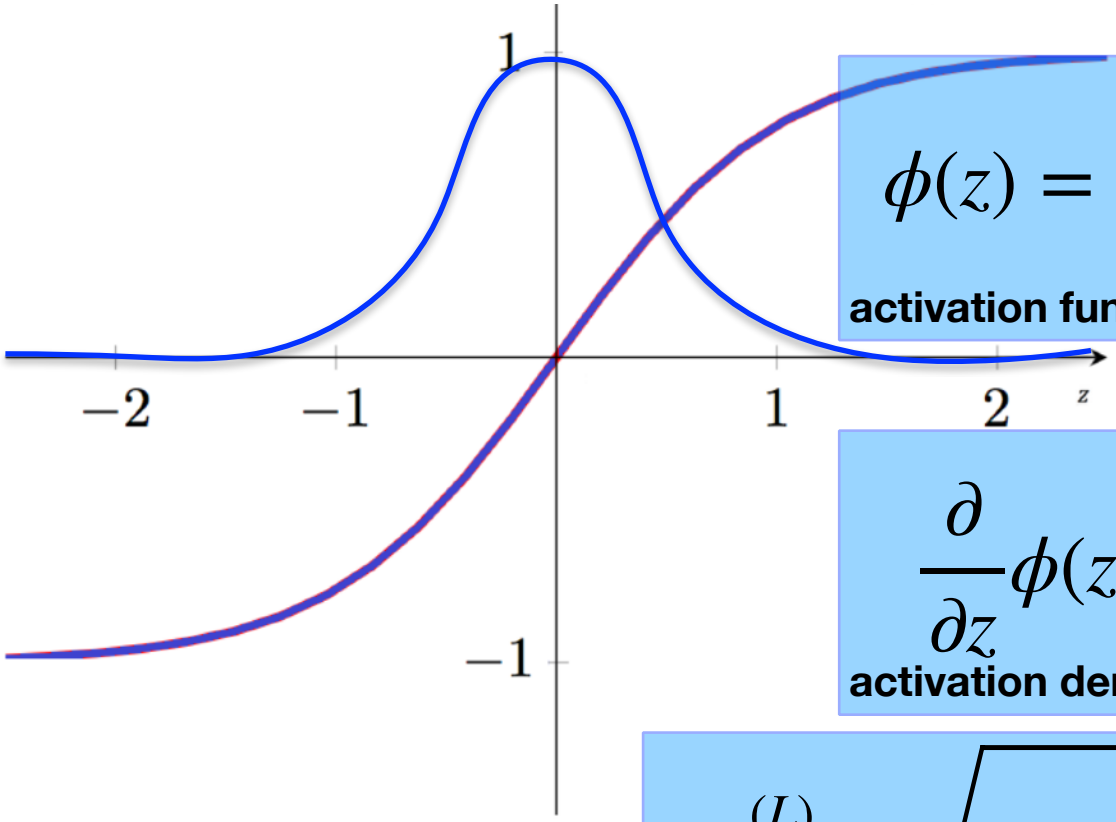


Can we do better than sigmoid?

- Perhaps! Sigmoid assumes [0, 1] firing, but that doesn't need to be true, right?
- If we change the activation, we need to:
  - derive the gradient for back prop
  - define what "saturation" means
  - update initialization of w_{ij} to not "saturate" in feedforward
  - update initialization so that sensitivity variance is about "1"
- Other researchers have done this for you! But you need to use it properly…

New Activation: Hyperbolic Tangent

- Basically a sigmoid from -1 to 1



$$\phi(z) = \frac{\sinh(z)}{\cosh(z)} = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

activation function

$$\frac{\partial}{\partial z}\phi(z) = \operatorname{sech}^2(z)$$

Back Prop

activation derivative

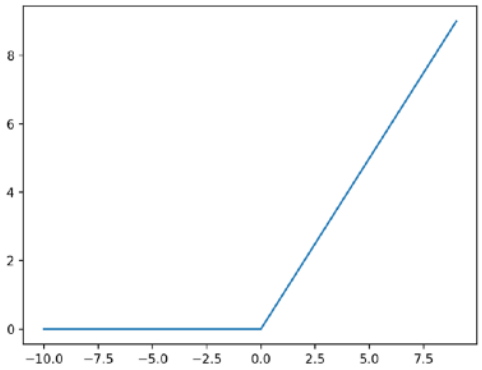
$$w_{ij}^{(L)} \sim \sqrt{\frac{2}{n^{(L)} + n^{(L+1)}}}$$

dense init with activation

Similar to sigmoid, ***no factor of 4***

New Activation: ReLU

- A new nonlinearity: **rectified linear units**



z

$$\phi(z) = \begin{cases} z, \text{ if } z > 0 \\ 0, \text{ else} \end{cases}$$

activation function

$$\frac{\partial}{\partial z}\phi(z) = \begin{cases} 1, \text{ if } z > 0 \\ 0, \text{ else} \end{cases}$$

activation derivative

it has the advantage of **large gradients** and **extremely simple** derivative

```
class Relu(Layer):
    def __call__(self,input):
        self.input = input
        self.output = np.maximum(input, 0)
        return self.output

    def backward(self, sensitivity_in):
        return (self.output > 0) * sensitivity_in
```

$$w_{ij}^{(L)} \sim \sqrt{2}\sqrt{\frac{2}{n^{(L)} + n^{(L+1)}}}$$

dense init with activation

Other Activation Functions

- Sigmoid Weighted Linear Unit **SiLU** (also called Swish)
- Mixing of sigmoid, σ, and ReLU

$$\phi(z) = \sigma(z) \cdot z$$

activation function

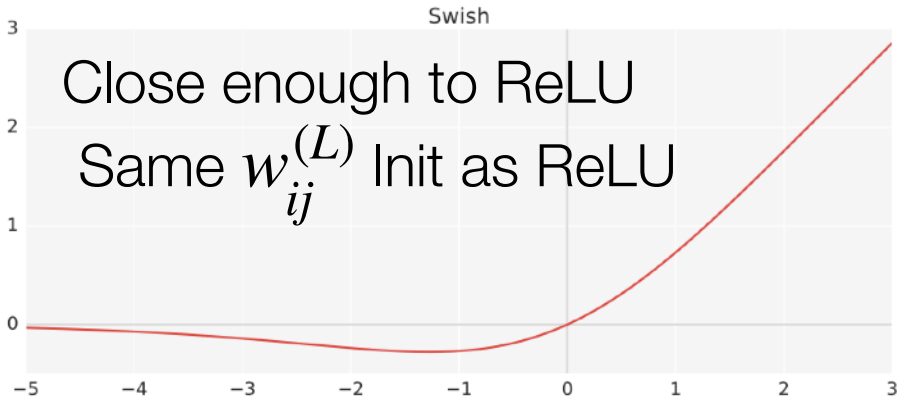


Figure 1: The Swish activation function.

$$\frac{\partial \phi(z)}{\partial z} = \frac{\partial}{\partial z}\sigma(z) \cdot z$$

$$= z \cdot \left[\frac{\partial}{\partial z}\sigma(z)\right] + \sigma(z) \cdot \left[\frac{\partial}{\partial z} z\right]$$

$$= z \cdot \sigma(z)(1 - \sigma(z)) + \sigma(z)$$

$$= z \cdot \sigma(z) + \sigma(z) \cdot (1 - z \cdot \sigma(z))$$

$$= \phi(z) + \sigma(z) \cdot (1 - \phi(z))$$

activation derivative

Elfwing, Stefan, Eiji Uchibe, and Kenji Doya. "Sigmoid-weighted linear units for neural network function approximation in reinforcement learning." Neural Networks (2018).

Ramachandran P, Zoph B, Le QV. Swish: a Self-Gated Activation Function. arXiv preprint arXiv:1710.05941. 2017 Oct 16

Glorot and He Initialization

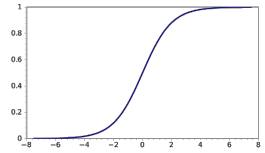
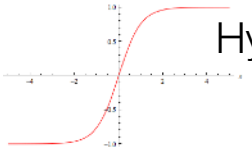
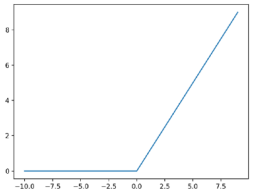
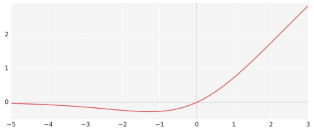
We have solved this assuming the activation output is in the range -4 to 4 (for a sigmoid) and assuming that we use Gaussian for sampling.

This range is different depending on the activation and assuming Gaussian or Uniform sampling.

	Uniform	Gaussian
Tanh	$w_{ij}^{(L)} \sim \sqrt{\frac{6}{n^{(L)} + n^{(L+1)}}}$	$w_{ij}^{(L)} \sim \sqrt{\frac{2}{n^{(L)} + n^{(L+1)}}}$
Sigmoid	$w_{ij}^{(L)} \sim 4\sqrt{\frac{6}{n^{(L)} + n^{(L+1)}}}$	$w_{ij}^{(L)} \sim 4\sqrt{\frac{2}{n^{(L)} + n^{(L+1)}}}$
ReLU SiLU	$w_{ij}^{(L)} \sim \sqrt{2}\sqrt{\frac{6}{n^{(L)} + n^{(L+1)}}}$	$w_{ij}^{(L)} \sim \sqrt{2}\sqrt{\frac{2}{n^{(L)} + n^{(L+1)}}}$

Summarized by Glorot and He

Activations Summary

	Definition	Derivative	Weight Init (Uniform Bounds)
 Sigmoid	$\phi(z) = \frac{1}{1+e^{-z}}$	$\nabla\phi(z) = a(1-a)$	$w_{ij}^{(L)} \sim \pm 4\sqrt{\frac{6}{n^{(L)}+n^{(L+1)}}}$
 Hyperbolic Tangent	$\phi(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$\nabla\phi(z) = \frac{4}{(e^{z}+e^{-z})^2}$	$w_{ij}^{(L)} \sim \pm \sqrt{\frac{6}{n^{(L)}+n^{(L+1)}}}$
 ReLU	$\phi(z) = \begin{cases} z, \text{ if } z > 0 \\ 0, \text{ else} \end{cases}$	$\nabla\phi(z) = \begin{cases} 1, \text{ if } z > 0 \\ 0, \text{ else} \end{cases}$	$w_{ij}^{(L)} \sim \pm \sqrt{2}\sqrt{\frac{6}{n^{(L)}+n^{(L+1)}}}$
 SiLU	$\phi(z) = \frac{z}{1+e^{-z}}$	$\nabla\phi(z) = \phi(z) + \sigma(z)\cdot(1-\phi(z))$	

Activations Code Update

```
class Dense(SimpleDense):
    def __init__(self, input_size, output_size, init_type='GlorotUniform'):
        """Initialize weights with small random numbers."""
        super().__init__(input_size, output_size)

        if init_type == 'GlorotUniform':
            init_bound = 4*np.sqrt(6. / (output_size + input_size))
        elif init_type == 'HeUniform':
            init_bound = np.sqrt(2)*np.sqrt(6. / (output_size + input_size))
        else:
            init_bound = 1.0

        num_elems = output_size*input_size
        tmp_weights = np.random.uniform(-init_bound, init_bound, size=(output_size, input_size))

        self.weights = Parameter(tmp_weights)
        self.bias = Parameter(np.zeros((output_size, 1)))
```

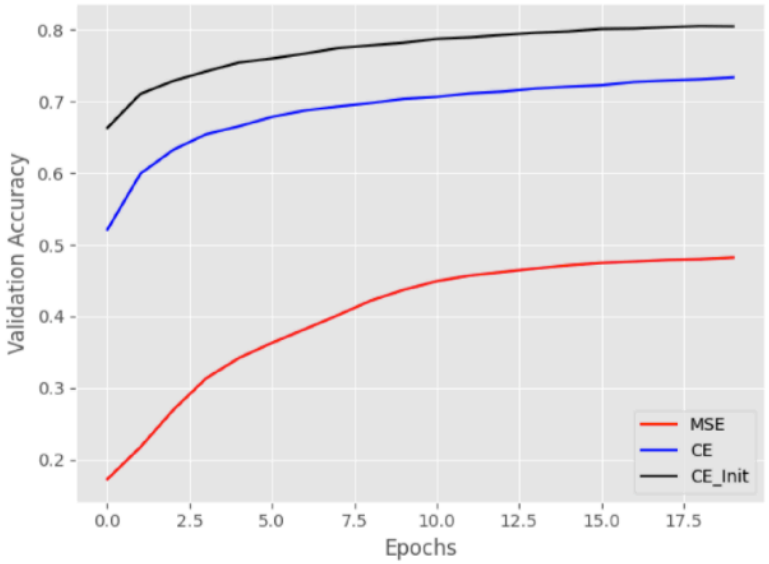
$$w_{ij}^{(L)} \sim 4\sqrt{\frac{6}{n^{(L)} + n^{(L+1)}}}$$

$$w_{ij}^{(L)} \sim \sqrt{2}\sqrt{\frac{6}{n^{(L)} + n^{(L+1)}}}$$

```
Model([Dense(784,256),
  Relu(),
  Dense(256,128),
  Relu(),
  Dense(128,10)])

Model([Dense(784,256),
  Sigmoid(),
  Dense(256,128),
  Sigmoid(),
  Dense(128,10)])
```

Three layer network, performance is essentially the same



CE+Init, Fashion MNIST

CE, Fashion MNIST

MSE, Fashion MNIST

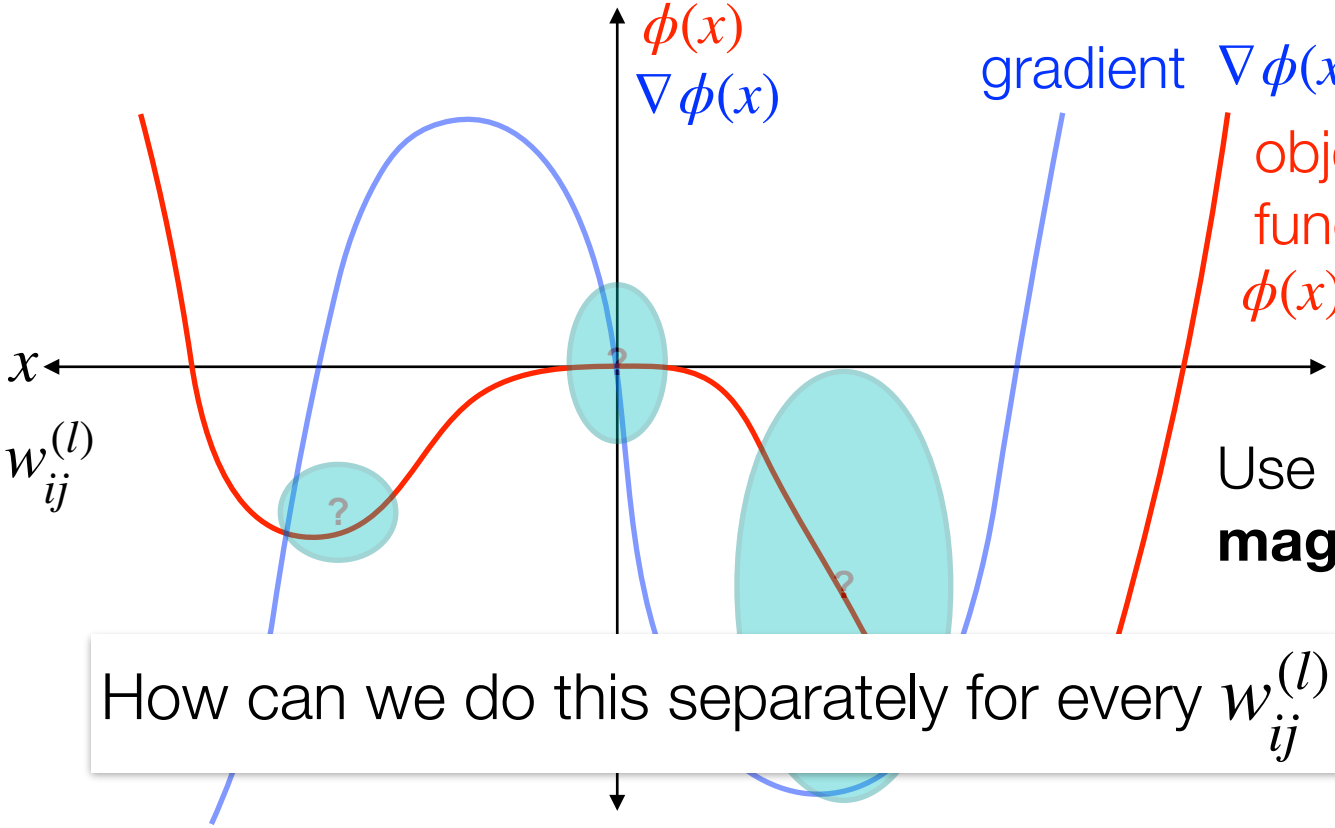
Adaptive Optimization



Going beyond
changing the learning rate

Be adaptive based on Gradient Magnitude?

- Decelerate down regions that are steep
- Accelerate on plateaus



gradient $\nabla\phi(x) = 16x^3 + 6x^2 + 24x$

objective function $\phi(x) = 4x^4 + 2x^3 + 12x^2$

Use **inverse** of **magnitude** of **gradient**!

How can we do this separately for every $w_{ij}^{(l)}$ in every $\mathbf{W}^{(l)}$?

Momentum on inverse magnitude: be robust to **abrupt changes** in **steepness** (accumulate momentum)

http://www.technologyuk.net/mathematics/differential-calculus/higher-derivatives.shtml

Be adaptive based on Gradient Magnitude?

Inverse magnitude of gradient in multiple directions?

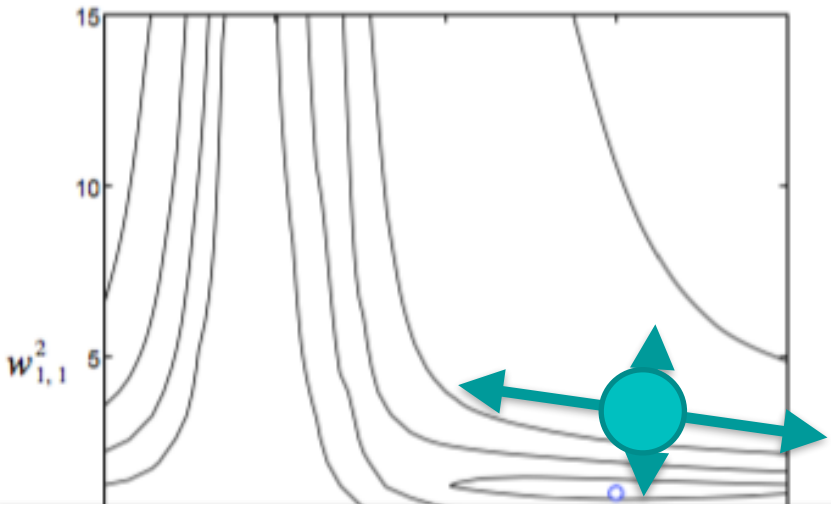
$$\mathbf{W}_{k+1} \leftarrow \mathbf{W}_k + \eta \frac{1}{\sqrt{\mathbf{G}_k + \epsilon}} \odot \nabla J(\mathbf{W}_k)$$

$$\mathbf{G}_k = \nabla J(\mathbf{W}_k) \odot \nabla J(\mathbf{W}_k)$$

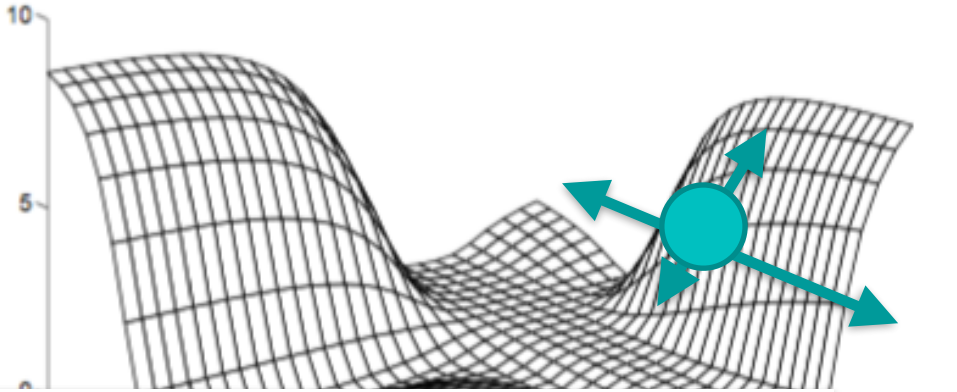
same size as $\mathbf{W}$

new matrix for normalizing

gradient




```
G = gradW1 * gradW1
W1 += eta*gradW1 / sqrt(G+eps)
```



Now we just need to add momentum to $\mathbf{G}_k^{(l)}$

Note: $\mathbf{G}$ exists for every layer, but we will abuse layer notation

AdaGrad Example

$$\mathbf{W}_{k+1} = \mathbf{W}_k - \eta \cdot \rho_k$$

- AdaGrad

all operations are per element

$$\mathbf{G}_k = \gamma \cdot \mathbf{G}_{k-1} + \nabla J(\mathbf{W}_k) \odot \nabla J(\mathbf{W}_k)$$

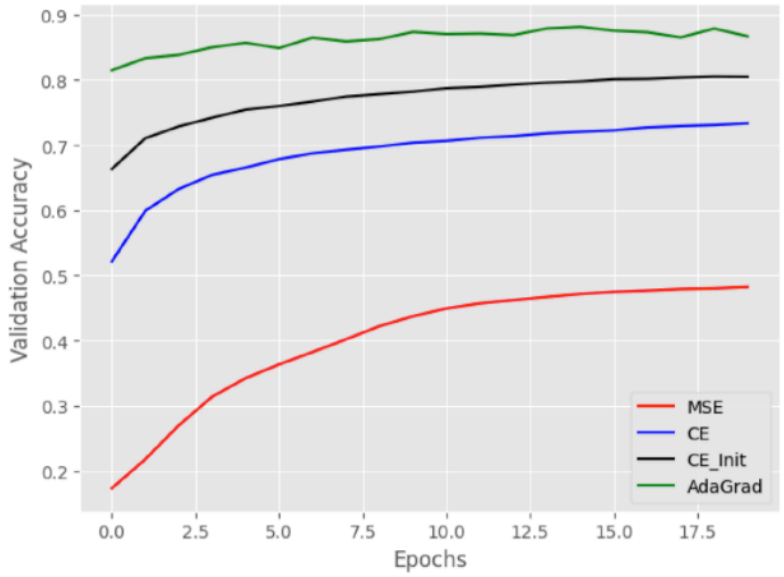
$$\rho_k = \frac{1}{\sqrt{\mathbf{G}_k + \epsilon}} \odot \nabla J(\mathbf{W}_k)$$



```
class AdaGrad(SGD):
    def step(self):
        # repurpose the momentum cache for storing G
        for param, G in zip(self.parameters, self._cache):
            G[:] = self.gamma*G + param.grad*param.grad
            param.value -= self.learning_rate * param.grad / np.sqrt(G + 1e-12)
```

cache stores G per trainable param

```
MSE Accuracy on Test Data: 0.4821
CE Accuracy on Test Data: 0.7334
CE_Init Accuracy on Test Data: 0.8046
AdaGrad Accuracy on Test Data: 0.8669
```



CE+Init+AdaGrad, Fashion MNIST
CE+Init, Fashion MNIST
CE, Fashion MNIST

MSE, Fashion MNIST

Starting to **saturate performance** on Fashion MNIST

Common Adaptive Strategies $\mathbf{W}_{k+1} = \mathbf{W}_k - \eta \cdot \rho_k$

Adjust each element of gradient by the steepness

- AdaGrad

$$\rho_k = \frac{1}{\sqrt{\mathbf{G}_k + \epsilon}} \odot \nabla J(\mathbf{W}_k)$$

all operations are per element

where

$$\mathbf{G}_k = \gamma \cdot \mathbf{G}_{k-1} + \nabla J(\mathbf{W}_k) \odot \nabla J(\mathbf{W}_k)$$

- RMSProp

$$\rho_k = \frac{1}{\sqrt{\mathbf{V}_k + \epsilon}} \odot \nabla J(\mathbf{W}_k)$$

all operations are per element

$$\hat{\mathbf{G}}_k = \nabla J(\mathbf{W}_k) \odot \nabla J(\mathbf{W}_k)$$

$$\mathbf{V}_k = \gamma \cdot \mathbf{V}_{k-1} + (1 - \gamma) \cdot \hat{\mathbf{G}}_k$$

- AdaDelta

$$\rho_k = \frac{1}{\sqrt{\mathbf{V}_k + \epsilon}} \odot \mathbf{M}_k$$

all operations are per element

$$\mathbf{M}_{k+1} = \gamma \cdot \mathbf{M}_k + (1 - \gamma) \cdot \nabla J(\mathbf{W}_k)$$

- AdaM — $\mathbf{G}$ updates with decaying momentum of J and J^2
- NAdaM — same as Adam, but with nesterov's momentum

- **None** of these are **"one-size-fits-all"** because the space of neural network **optimization varies** by problem, AdaM is **popular/stable**

AdaM

AdaDelta

$$\mathbf{V}_k = \gamma \cdot \mathbf{V}_{k-1} + (1-\gamma) \cdot \nabla J(\mathbf{W}_k) \odot \nabla J(\mathbf{W}_k)$$
$$\mathbf{M}_k = \gamma \cdot \mathbf{M}_{k-1} + (1-\gamma) \cdot \nabla J(\mathbf{W}_k)$$

Adam: a Method for Stochastic Optimization

ICLR 2015, Test of Time Paper 2025

D. Kingma and J. Ba



"Too incremental" scores too low to accept

$$\beta_1 = 0.9,\ \beta_2 = 0.999,\ \eta = 0.001,\ \epsilon = 10^{-8}$$
$$k = 0,\ \mathbf{M}_0 = \mathbf{0},\ \mathbf{V}_0 = \mathbf{0}$$

For each epoch:

update iteration $k \leftarrow k + 1$

get gradient $\nabla J(\mathbf{W}_k)$

for large k, $\hat{\mathbf{M}} \approx \mathbf{M},\ \hat{\mathbf{V}} \approx \mathbf{V}$

momentum gradient *like AdaDelta*

$$\mathbf{M}_k \leftarrow \beta_1 \cdot \mathbf{M}_{k-1} + (1-\beta_1) \cdot \nabla J(\mathbf{W}_k)$$

momentum squared gradient *like AdaDelta*

$$\mathbf{V}_k \leftarrow \beta_2 \cdot \mathbf{V}_{k-1} + (1-\beta_2) \cdot \nabla J(\mathbf{W}_k) \odot \nabla J(\mathbf{W}_k)$$

boost moments magnitudes (notice k in exponent)

$$\hat{\mathbf{M}}_k \leftarrow \frac{\mathbf{M}_k}{(1-[\beta_1]^k)} \qquad \hat{\mathbf{V}}_k \leftarrow \frac{\mathbf{V}_k}{(1-[\beta_2]^k)}$$

update gradient, normalized by second moment similar to AdaDelta

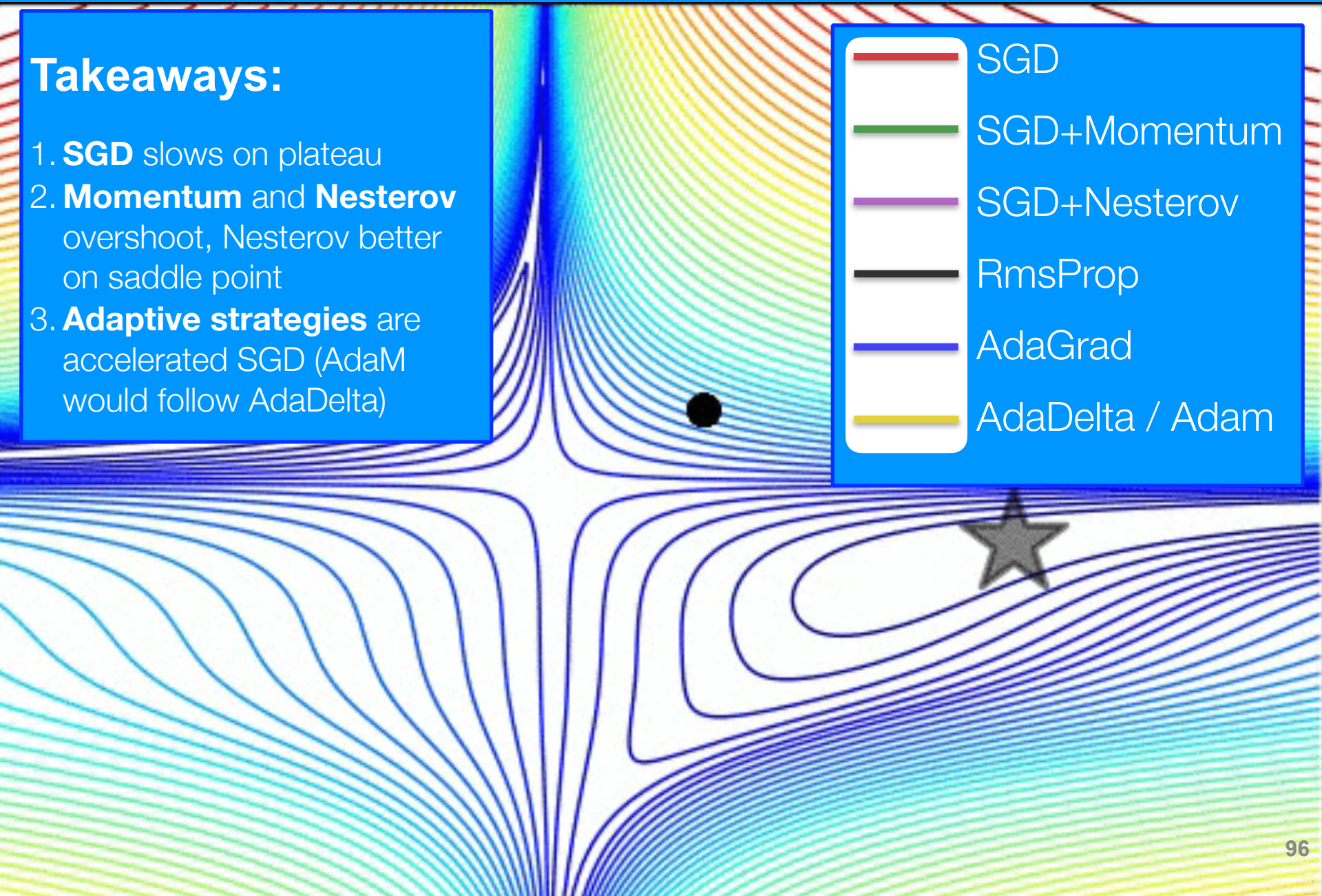
$$\mathbf{W}_k \leftarrow \mathbf{W}_{k-1} - \eta \cdot \frac{\hat{\mathbf{M}}_k}{\sqrt{\hat{\mathbf{V}}_k + \epsilon}}$$

gradient with momentum

squared magnitude normalizer

Visualization of Optimization

https://ruder.io/optimizing-gradient-descent/



Exam Format and Possible Content

- 15 multiple choice questions (50% credit)
- 3 multi-part, open-ended questions (50% credit)
- One sheet of notes, front/back
- No phones, devices, etc.
- Table Data, Numeric/Categorical, Visualization
- Missing Data, **Imputation**, Exploratory Data Analysis
- Logistic Regression, Techniques, Interpretation, *"What if we changed… ?"* **Decision Boundaries**
- Neural Networks: Notation, Optimization, Adaptive Techniques,
- **Drawing** a described network, with chain rule markup

End of Session

- **Next time is Wrap Up and Review**
- Everything covered thus far will be on the mid-term Exam